IA Arkanoid

Group assignment: build an agent that plays Arkanoid

Mário Antunes · Luís Seabra Lopes

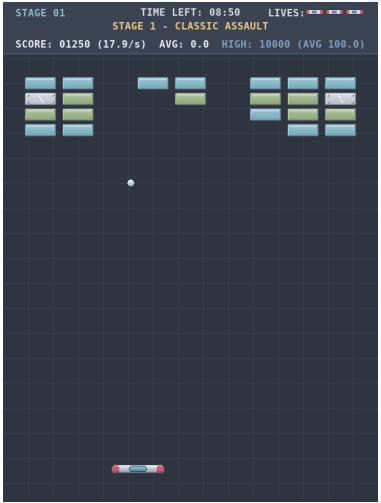
Artificial Intelligence 2026/2027
DETI · Universidade de Aveiro

12 October 2026

© 2026, Instituto de Telecomunicações · Universidade de Aveiro

The assignment in one slide

- Groups of **three** students; names and mechanographic numbers in the `README.md` of the group repository
- Build an **agent** (`student.py`) that plays a variant of *Arkanoid* by itself
- Clear all levels, collect points, **do not lose the ball**
- Performance is measured as **points per second** on a sequence of maps
- Python 3.12 (3.10 minimum), only the libraries of `pyproject.toml`
- **1st delivery: 18 November 2026 · final delivery: 16 December 2026**



Setup 1/4: accept the assignment (classroom50)

classroom50 sets up the **project** and the **group**:

`https://classroom50.org/detiuaveiro/inteligencia-artificial/assignments/arkanoid/accept`

1. Agree the **group of three** first
2. Open the link above and **accept** the assignment
3. **Create** the group (one student) and **join** it (the others): one group, one repository
4. You get the **group repository** with the game code: this is where you work and deliver
5. Write names and mechanographic numbers in its `README.md`

Setup 2/4: get the code

1. Open **your group repository** (created in step 1) and copy its URL
2. Clone it and enter the folder; **every command below runs from this folder**

```
git clone <URL of your group repository>
cd <repository folder>
```

3. Add the course template as `upstream`: this is how you receive bug fixes, new levels and updates

```
git remote add upstream \
    https://github.com/detiuaveiro/ia-arkanoid-template
```

Setup 3/4: the Python environment

1. Create a virtual environment, **once** (Python 3.12; 3.10 is the minimum)
2. **Activate** it, in **every** terminal you use
3. Install the requirements: `pip install .` (reads `pyproject.toml`; there is no `requirements.txt`)
4. Check the installation

```
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install .
python -c "import aigf, numpy, pygame, websockets"
```

Creating or activating a venv installs nothing: step 3 is not optional. No error from the last line = ready.

Setup 4/4: run the game (three terminals)

Activate the venv in each one, then start them **in this order**:

Terminal	Role	Command
1	Server (the game)	`python -m server.server`
2	Viewer (the window)	`python -m viewer.viewer`
3	Your agent	`python -m agents.dummy_agent --id a12345`

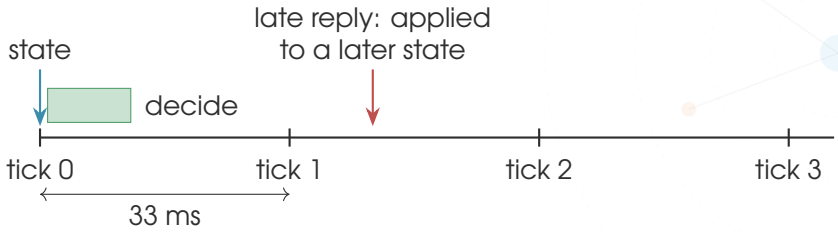
- The game starts when an agent connects; when it ends the server waits for a new agent
- By hand: `python -m agents.manual_agent --id a12345` (keys A / D or arrows, Q quits)
- Your agent: `python student.py --id <your number>`

Architecture: who talks to whom



- WebSocket, JSON: the **state** (paddle, ball, bricks, lives, score, time) and your **action**
- The action is `{"action": "move", "direction": "WEST"}` or `"EAST"`; nothing = the paddle stays
- You write `deliberate()`: it reads `self.current_state` and returns an action or `None`

Timing: you do not get unlimited time to think



- **30 ticks per second**: one state every **33 ms**; the server **does not wait**
- An action moves the paddle **28 px** on arrival; no answer = no move
- Keep `deliberate()` fast (a ball tracker needs microseconds)
- `--acceleration` above ~20 makes replies late: develop with it, do not judge with it

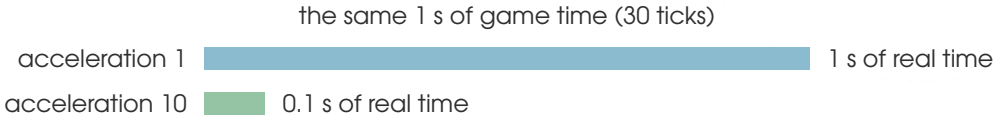
Time and speed: `--fps` and `--acceleration`

`--fps` (default **30**)

- One **tick** = $1/\text{fps}$ s of *game time*
- Each tick: one state sent, one action possible
- It defines the game (actions per second): **do not change it**

`--acceleration` (default **1**)

- How fast the game runs in *real time*
- The game itself is **unchanged**: same ticks, same physics, same 600 s limit
- 10 = one second of game time in 0.1 s of real time



At acceleration X your agent receives $30 \times X$ states per real second: it has only $33/X$ ms to decide.

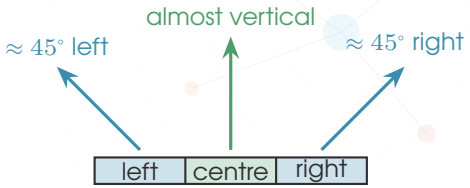
What acceleration means for your agent

`--acceleration`	States / real s	Time to decide	Use
1 (default)	30	33 ms	**judge here**
5	150	6.7 ms	develop
10	300	3.3 ms	many games
20	600	1.7 ms	upper limit
60	1800	0.6 ms	replies late

- Same game, same physics: a game takes 600 s of **game time** per level at any acceleration; with 10 it ends in 60 s of real time
- Too fast for your agent = your moves are applied to a later state: you play worse than at speed 1
- Always confirm a result at `--acceleration 1` before trusting it

The game rules that matter

- Field 600×800 px; bricks 50×20; paddle 84×14; ball speed 400 px/s
- Brick points (only when it **breaks**): red 50, blue 100, green 150, yellow 200; metal is indestructible
- **3 lives** per game; a level ends when no destructible brick is left
- Each level has **600 s** of game time; not cleared in time = game over
- The paddle has **three zones**: where the ball hits decides where it goes



A good agent does not only **follow** the ball: it **aims** it.

`student.py`: what you deliver and how it runs

- The agent is the module `student.py` in your repository (subclass of `BaseAgent`, implement `deliberate()`)
- It is run as `python student.py --id <your mechanographic number>`; `--id` (or the `AGENT_ID` variable) identifies the group in the dashboard
- It is evaluated in **inference mode**: only the libraries declared in `pyproject.toml` exist (no PyTorch, scikit-learn, …)
- A model you trained elsewhere must be saved as **files in the repository** and run with those libraries

Do not

- rely on libraries that are not in `pyproject.toml`
- change the server, the viewer or the base agent
- start without an `--id`: it cannot be ranked

Scoring: the number that is graded

- `normalized_score` of a level = points / seconds of that level, **only if the level is cleared** (else 0)
- `avg_norm_score` = mean over the levels of the sequence (0 if lost or not reached): the **performance**
- Raw `score` only breaks ties

Game	Level 1	Level 2	Level 3
Points / time	2750 in 200 s	lost	2550 in 400 s
`normalized_score`	13.8	0	6.4

`avg_norm_score` $= (13.8 + 0 + 6.4)/3 \approx 6.7$ points/s. **Clear first, then be fast.**

How it is marked

- A **sequence of maps**: the 3 published levels plus **additional unpublished maps**; 3 lives for the whole game, 600 s per level
- Each agent plays **10 games** of the sequence; the average is mapped linearly to marks:

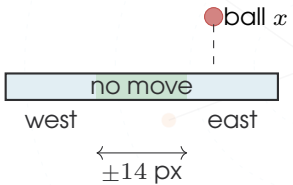
0 points/s	10 points/s	class median	class maximum
0 / 20	10 / 20	16 / 20	20 / 20

- **1st delivery (18 Nov):** agent code only
- **2nd delivery (16 Dec):** performance 90%, design 7.5% (`presentation.pdf`, at most 5 pages), implementation 2.5%
- 5% per late day; accepted pull requests: up to +1 / 20

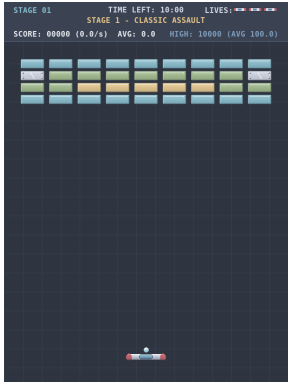
Baselines: tracker v1 and tracker v2

v1 (`tracker_v1.py`): moves towards the ball x; ball over the **centre third** of the paddle: no move
v2 (`tracker_v2.py`): the same with a dead band of **half a paddle step** (14 px): the paddle settles

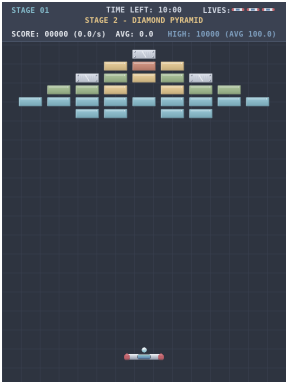
- No prediction, no bricks, no aiming
- 20 games per level (600 s): level 1 cleared 16/20, level 2 20/20, **level 3 only 7/20**
- A floor, not the goal: use the velocity, predict the landing, choose the paddle zone



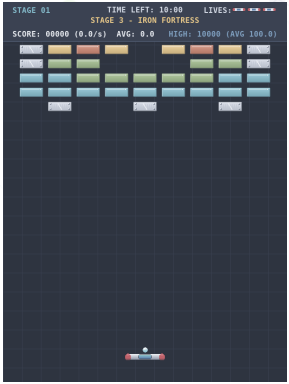
The current maps



Stage 1: Classic Assault



Stage 2: Diamond Pyramid



Stage 3: Iron Fortress

Metal bricks only deflect the ball. The evaluation also uses **unseen** maps.

The code and the guide are alive

- Bugs, tweaks, **new levels** and the **dashboard** (ranking by `--id`) will arrive during the semester
- It is **your responsibility** to pull them: template as `upstream`, fetch and merge often
- Commit before merging; keep your work in **your own files**, take upstream's game files
- Follow `#ai` on Slack, e-mail and e-learning; the guide (EN/PT) is updated too
- Compare versions with several games: the launch angle is random

```
git remote add upstream <template repository URL>
git fetch upstream && git merge upstream/main
```

Do not touch: `server/`, `viewer/`, `agents/base_agent.py`

- These files are updated by us: if you edit them, **every update becomes a merge conflict** for your group
- The server is the referee: your agent is judged against the original rules
- Create new files and folders for your work: `student.py` and anything it needs
- Found a bug or have an improvement for the game? Open a **pull request** to the template: if accepted, bonus up to 1 point (in 20)

Summary

Do

- Run the quick start, play with the manual agent
- Measure a tracker, then beat it
- Use the 30 Hz tick budget wisely
- Pull the updates, follow `#ai`

Dates and rules

- 1st delivery: **18 November 2026**
- Final: **16 December 2026**
- Python 3.12, `pyproject.toml` libraries only
- Marks: 10 points/s = 10/20, median 16/20

Questions: Slack `#ai` · assignment: classroom50, `inteligencia-artificial` / `arkanoid`